



Module 3: AI for Mechatronics

Autoencoders: Learning Useful Representations Without Labels

Dr. Rajan Prasad

SMEC
VIT University

Fall 2026

The task

A production line captures thousands of images of metal surfaces every day. Most products are normal; only a few contain scratches, cracks, or dents.

The difficulty

Labelling every image requires expert time, and examples of every possible defect do not exist.

A different question

Instead of first teaching a model every defect, can it learn what a normal product looks like?

If it can reconstruct normal surfaces accurately, an unusual surface should be harder to reconstruct.

Can a network learn the essential structure of data by trying to reproduce its own input?

The Suitcase Challenge: Keep What Matters

At home

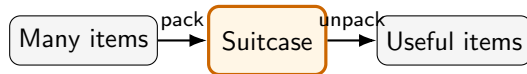
You have clothes, books, tools, and many personal items.

During travel

Everything must fit into one small suitcase. You cannot carry every detail, so you retain the most useful items.

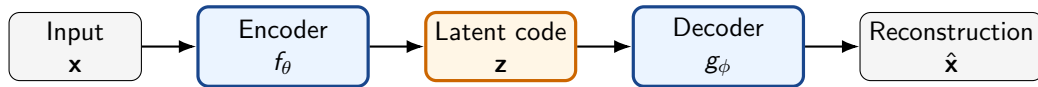
After arrival

You unpack the suitcase and recover an approximate version of what you need.



The small suitcase forces compression; an autoencoder uses a bottleneck for the same purpose.

An Autoencoder Learns to Reconstruct Its Input



Encoder Function: f_θ

$$\mathbf{z} = f_\theta(\mathbf{x}) = \sigma_e(\mathbf{W}_e \mathbf{x} + \mathbf{b}_e)$$

- Converts the input into a compact representation.
- $\theta = \{\mathbf{W}_e, \mathbf{b}_e\}$ represents the trainable encoder parameters.
- $\mathbf{z}$ retains the most useful input features.

Decoder Function: g_ϕ

$$\hat{\mathbf{x}} = g_\phi(\mathbf{z}) = \sigma_d(\mathbf{W}_d \mathbf{z} + \mathbf{b}_d)$$

- Reconstructs the input from the latent code.
- $\phi = \{\mathbf{W}_d, \mathbf{b}_d\}$ represents the trainable decoder parameters.
- $\hat{\mathbf{x}}$ should be as close as possible to $\mathbf{x}$.

$$\underbrace{\mathbf{x} \xrightarrow{f_\theta} \mathbf{z} \xrightarrow{g_\phi} \hat{\mathbf{x}}}_{\hat{\mathbf{x}} = g_\phi(f_\theta(\mathbf{x}))}$$

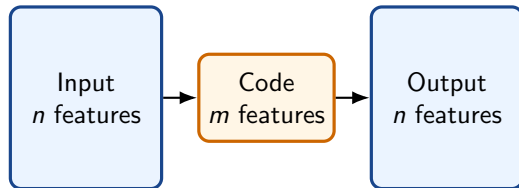
During training, the input is also the target: $\mathbf{y} = \mathbf{x}$. Minimizes the difference between $\mathbf{x}$ and $\hat{\mathbf{x}}$.

Suppose the input has n features and the latent code has m features:

$$\mathbf{x} \in \mathbb{R}^n, \quad \mathbf{z} \in \mathbb{R}^m, \quad m < n.$$

Undercomplete representation

The network must identify patterns that are most useful for reconstruction because the latent space has lower dimension.



A bottleneck alone is not always sufficient; model capacity and regularization also matter.

For continuous, normalized features, a common loss is mean squared error:

$$L_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2.$$

Large differences between input and reconstruction produce a larger penalty.

For binary-valued pixels or features, binary cross-entropy may be used:

$$L_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i)].$$

Training adjusts both encoder and decoder parameters to minimize reconstruction loss.

Encode

$$\mathbf{a}^{(e)} = W_e \mathbf{x} + \mathbf{b}_e,$$
$$\mathbf{z} = \sigma(\mathbf{a}^{(e)}).$$

Decode

$$\mathbf{a}^{(d)} = W_d \mathbf{z} + \mathbf{b}_d,$$
$$\hat{\mathbf{x}} = \sigma(\mathbf{a}^{(d)}).$$

Training objective

$$\min_{\theta, \phi} \frac{1}{M} \sum_{j=1}^M L(\mathbf{x}^{(j)}, g_{\phi}(f_{\theta}(\mathbf{x}^{(j)}))).$$

- 1 Encode $\mathbf{x}$ to obtain $\mathbf{z}$.
- 2 Decode $\mathbf{z}$ to obtain $\hat{\mathbf{x}}$.
- 3 Calculate $L(\mathbf{x}, \hat{\mathbf{x}})$.
- 4 Propagate the reconstruction error backward.
- 5 Update $W_e, \mathbf{b}_e, W_d, \mathbf{b}_d$.

Gradient descent

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}, \quad \phi \leftarrow \phi - \eta \frac{\partial L}{\partial \phi}.$$

The error passes through the decoder, the latent code, and then the encoder.

An autoencoder uses ordinary backpropagation; no recurrence through time is required.

Use a two-feature input and one latent neuron:

$$\mathbf{x} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad W_e = \begin{bmatrix} 0.8 & -0.4 \end{bmatrix}, \quad b_e = 0.$$

With sigmoid activation,

$$z = \sigma(0.8) = 0.6900.$$

Let

$$W_d = \begin{bmatrix} 1.0 \\ -1.0 \end{bmatrix}, \quad \mathbf{b}_d = \mathbf{0}.$$

Then

$$\hat{\mathbf{x}} = \sigma(W_d z) = \begin{bmatrix} 0.6660 \\ 0.3340 \end{bmatrix}.$$

$$L_{\text{MSE}} = \frac{(1 - 0.6660)^2 + (0 - 0.3340)^2}{2}$$

$$L_{\text{MSE}} = 0.1116.$$

Supervised ANN

$$\mathbf{x} \longrightarrow \hat{\mathbf{y}}$$

Learns from labelled input–target pairs $(\mathbf{x}, \mathbf{y})$.

- Goal: classification or regression
- Target: external label or measured value
- Output: class, probability, or estimate

Autoencoder

$$\mathbf{x} \longrightarrow \mathbf{z} \longrightarrow \hat{\mathbf{x}}$$

Learns by reconstructing the input; explicit class labels are not required.

- Goal: representation learning
- Target: the original input
- Output: reconstruction and latent features

An autoencoder is still a neural network; the difference is its architecture and training objective.

ANN, RNN, and Autoencoder Solve Different Problems

Aspect	Feedforward ANN	RNN	Autoencoder
Main purpose	Classification or regression	Learn temporal relationships	Learn a compact representation
Information flow	Input to output	Recurrent state across time	Encoder to latent code to decoder
Typical target	Label or numerical value	Label or value associated with a sequence	Original input itself
Memory	No internal temporal state	Hidden state carries past context	No temporal memory in the basic form
Typical input	Fixed feature vector	Ordered sequence	Vector, image, or signal segment
Example	Classify a component	Predict motor condition from sensor history	Detect unusual vibration patterns

The choice depends on the learning objective, not simply on which model is more advanced.

Similar inputs often receive nearby latent representations:

$$\mathbf{x}_1 \approx \mathbf{x}_2 \quad \Rightarrow \quad f_{\theta}(\mathbf{x}_1) \approx f_{\theta}(\mathbf{x}_2).$$

The latent variables may capture dominant shape, operating condition, texture, or signal pattern.

Useful consequences

- fewer features for later models;
- visualization of high-dimensional data;
- clustering of similar operating conditions;
- comparison through latent-space distance.

Latent coordinates are learned features; they do not automatically have a simple physical interpretation.

A corrupted input $\tilde{\mathbf{x}}$ is supplied to the encoder, but the clean sample $\mathbf{x}$ remains the target:

$$\tilde{\mathbf{x}} \rightarrow f_{\theta} \rightarrow \mathbf{z} \rightarrow g_{\phi} \rightarrow \hat{\mathbf{x}}.$$

$$L = L(\mathbf{x}, \hat{\mathbf{x}}).$$

Why it helps

The network cannot succeed by copying every input fluctuation. It must learn stable patterns that help recover the clean data.

Examples

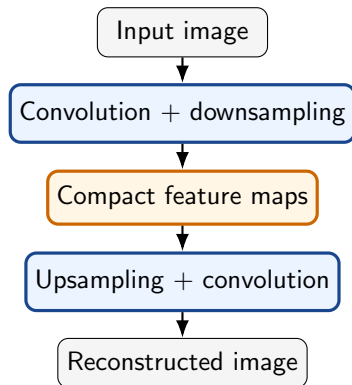
Sensor noise removal, image restoration, and robust feature learning.

Encoder

Convolution and downsampling convert an image into compact feature maps.

Decoder

Upsampling and convolution reconstruct the image at its original resolution.



Useful for surface inspection, visual denoising, and image anomaly detection.

An RNN, LSTM, or GRU can serve as the encoder and decoder when the input is a time sequence:

$$(\mathbf{x}_1, \dots, \mathbf{x}_T) \rightarrow \mathbf{z} \rightarrow (\hat{\mathbf{x}}_1, \dots, \hat{\mathbf{x}}_T).$$

Mechatronics example

Compress a window of motor current and vibration measurements, then reconstruct the complete window.

Important distinction

“Autoencoder” describes the reconstruction objective; “RNN” describes a network structure for sequential data.

- 1 Train primarily on normal operating data.
- 2 Reconstruct a new sample.
- 3 Calculate its anomaly score:

$$s(\mathbf{x}) = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2.$$

- 4 Flag the sample when $s(\mathbf{x}) > \tau$.

Expected behaviour

Normal patterns are reconstructed well, whereas sufficiently different patterns may produce larger errors.

Caution

The threshold must be validated. A powerful autoencoder may also reconstruct some anomalies well.

Machines and manufacturing

- bearing and gearbox anomaly detection;
- surface-defect inspection;
- sensor fault detection;
- dimensionality reduction for monitoring.

Robotics and autonomous systems

- visual and LiDAR feature compression;
- denoising of sensor measurements;
- unusual-behaviour detection;
- compact state representation for control.

Autoencoders are most useful when compact features or normal-pattern modelling matter more than direct labels.

Data representation

Compression, visualization, clustering, and feature extraction.

Cybersecurity and finance

Unusual network traffic, transaction patterns, and system behaviour.

Images and signals

Denoising, restoration, missing-data recovery, and anomaly detection.

Healthcare

Representation of medical images and physiological signals, with expert validation.

Good reasons

- labels are scarce but data are abundant;
- the input is high-dimensional;
- normal data are easier to obtain than fault data;
- a learned representation is required.

Questions to ask

- What should be reconstructed?
- What loss matches the data?
- How large should the latent space be?
- How will reconstruction quality be validated?

A low reconstruction loss is not enough; the learned representation must support the intended task.

- Scale continuous features consistently.
- Separate training, validation, and test data.
- Match output activation and loss to the data range.
- Tune latent dimension and model capacity.
- Monitor both training and validation loss.
- Inspect reconstructed examples, not only average loss.
- Choose anomaly thresholds using validation data.
- Test robustness across operating conditions.

Learning path

$$\mathbf{x} \xrightarrow{\text{encoder}} \mathbf{z} \xrightarrow{\text{decoder}} \hat{\mathbf{x}}$$

The encoder retains information needed by the decoder.

Training objective

$$\min_{\theta, \phi} L(\mathbf{x}, g_{\phi}(f_{\theta}(\mathbf{x}))) .$$

Reconstruction error guides ordinary backpropagation.

Scan to Open: Colab Notebook



An autoencoder learns a useful latent representation by reconstructing the input.

Instructions

- Scan the QR code to access the quiz.
- Sign in using your **VIT Google account**.
- The quiz contains **10 questions**.
- Only **one submission per account** is allowed.

QUIZ: Scan to begin



Autoencoders Learn What to Keep

Questions?